

AI DEV PROJECT - COMPLETE VIVA HANDBOOK

AgentForge එක මුල සිට අවසානය දක්වා

SRS handoff එක ලැබුණු නැත සිට UI, Planner, Design, Builder, Analyzer, QA සහ Bug
Fixer එකට වැඩ කරලා full app එක build වෙන ආකාරය

භාෂාව	සිංහල - technical නාම English වලින්
ඉලක්කය	Code දන්නේ නැති කෙනෙකුටත් viva එකට කියන්න පුළුවන් වීම
Scope	SRS internal generation සහ Deploy internal process නොව, approved SRS handoff සිට finished build දක්වා
Code	Source code lines නැහැ, file, function, prompt, data-flow සහ වගකීම පැහැදිලි කරයි

Workspace snapshot: 2026-08-23 | Teaching-only review | No application source was edited

මේ PDF එක භාවිතා කරන හොඳම ක්‍රමය

- මුලින් 2 සහ 3 පරිච්ඡේද කියවලා system එකේ big picture එක ගන්න.
- ඊළඟට 4 සිට 10 දක්වා build flow එක පිළිවෙලින් කියවන්න.
- ඔබ කළ agent/server කොටස සඳහා 11, 12, 13 සහ 14 පරිච්ඡේද විශේෂයෙන් පාඩම් කරන්න.
- අවසානයේ viva ප්‍රශ්න සහ short answers කොටසෙන් කටපාඩම් නොකර අදහසින් පිළිතුරු දීමට පුහුණු වන්න.
- Path එකක් හෝ function එකක් දුටු විට ඒක ලියන්නේ මොන file එකේද, ඒකෙන් සිදු වෙන්නේ කුමක්ද කියලා එක වාක්‍යයකින් කියන්න පුළුවන් වෙන්න.

ඔබ viva එකේ කියන මූලික වාක්‍යය

AgentForge කියන්නේ user ගේ approved requirements අරගෙන, ඒවා build plan එකකට හැරවලා, Next.js full-stack app එකේ files ලියලා, compile, static analysis, unit testing, browser testing සහ evidence-based repair කරන local-first AI software factory එකක්.

අන්තර්ගතය

අංකය	පරිච්ඡේදය
01	වචන 30කින් මුළු system එක
02	Project එකේ ප්‍රධාන කොටස්
03	App එක start වෙන හැටි
04	Ports සහ connections
05	UI එක build command එක සකස් කරන හැටි
06	SRS handoff එකේ සිට project folder එකට
07	Backend command එක receive කරන හැටි
08	run_agent_pipeline: සම්පූර්ණ timeline
09	Planner agent එක වැඩ කරන හැටි
10	Design role සහ UI contract
11	Builder agent එක app files ලියන හැටි
12	Generated app scaffold සහ technology stack
13	Analyzer: code කියවලා plan එකට සසඳන inspector
14	Bug Fixer: error එකෙන් root cause එකට
15	QA: unit tests, browser journeys සහ final quality
16	Model layer: Planner, Design, Builder සහ QA models
17	MongoDB සහ generated app data flow
18	Live UI feedback: backend event එක screen එකට
19	Generated project එකේ files නියෙන නැන්
20	Agent folder එකේ සම්පූර්ණ map එක
21	Planner, gates, repair සහ testing files map
22	Feature, picture, selection සහ server modules map
23	Prompt inventory
24	Error සහ edge cases
25	Viva questions සහ short answers
26	1-minute සහ 3-minute viva scripts
27	Glossary - ඉතා සරල අර්ථ
28	අවසාන මතක පත්‍රය

වචන 30කින් මුළු system එක

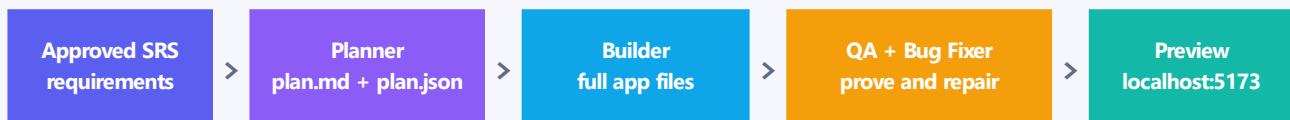
මුලින්ම viva එකේ opening answer එක. මෙය මතක තබාගන්නොත් පසුව technical details එකට පහසුවෙන් යන්න පුළුවන්.

30-second answer

User app idea එක හෝ approved SRS handoff එක UI එකෙන් build message එකක් ලෙස Python backend එකට යනවා. Planner requirements වලින් exact plan එකක් හදනවා. Design role look-and-feel contract එක හදනවා. Builder Next.js, React, Tailwind සහ MongoDB app files ලියනවා. Analyzer සහ QA code, routes, database behavior, unit tests සහ real browser journeys පරීක්ෂා කරනවා. Error එකක් ලැබුණොත් Bug Fixer evidence බලලා අදාළ files පමණක් repair කර නැවත test කරනවා. අවසානයේ app එක port 5173 මත preview වෙනවා.

Non-technical උපමාව

මෙය ගෙයක් හදන කණ්ඩායමක් ලෙස සිතන්න. SRS handoff එක customer අනුමත කළ requirement book එක. Planner architect drawing එක හදනවා. Design role interior design rules තීරණය කරනවා. Builder mason සහ engineer වගේ files හදනවා. Analyzer building inspector වගේ plan එකට ගැලපෙනවාද බලනවා. QA කෙනෙක් නිවසේ දොරවල්, taps සහ switches ඇත්තටම භාවිතා කර බලනවා. Bug Fixer fault එක නියෙන නැත ඥායා අදාළ කොටස පමණක් හදනවා. Studio UI එක customerට වැඩේ live පෙන්වන control room එක.



මෙම guide එකේ scope එක

- SRS agent එක requirements හදන අභ්‍යන්තර interview/generation logic එක විස්තර කරන්නේ නැහැ; approved handoff ලැබුණු තැනින් පටන් ගනී.
- Deployment agent එක cloud එකට publish කරන අභ්‍යන්තර logic එක විස්තර කරන්නේ නැහැ; finished build එකේ boundary එකේ නවතී.
- AgentForge Studio UI, Python server, agent folders, model routing, generated app, testing සහ repair path එක සම්පූර්ණයෙන් ඇතුළත්.
- Source code lines නොපෙන්වයි. ඒ වෙනුවට function එකේ business meaning, input, output, location සහ next connection එක පැහැදිලි කරයි.

Project එකේ ප්‍රධාන කොටස්

Folder එකක් එක department එකක් වගේ. මේ map එක project එකේ responsibility separation එක පෙන්වයි.

Location	Department	සරල වගකීම
desktop/	Electron desktop shell	Python backend සහ Next.js Studio start කර එක desktop window එකක පෙන්වයි.
studio/	Control-room UI	Prompt, model selection, build progress, preview, code view, testing view සහ edit tools.
backend/ server.py	Public backend entry	Async Python runtime එක ආරම්භ කරන ඉතා කුඩා stable endpoint එක.
backend/ server_runtime.py	Runtime assembler	Focused server modules නිශ්චිත order එකකට එක shared namespace එකකට load කරයි.
backend/ server_modules/	Server orchestration	HTTP, WebSocket, build pipeline, QA stages, repair සහ process lifecycle.
backend/ agents/	AI/domain agents	Planner, Builder, Analyzer, Bug Fixer, data, testing, feature editing සහ model client logic.
backend/ qa_agent/	Testing engine	Unit test authoring, Vitest runner, Playwright E2E, evidence, debugger සහ QA report.
backend/ srs-agent/	SRS sidecar	මෙම guide එකේ boundary: approved builder handoff එක ලබා දෙයි.
backend/ deployment-agent/	Deploy sidecar	මෙම guide එකෙන් පිටත: finished app එක cloud target එකකට deploy කරයි.
backend/ production-ready/	Generated projects	Default ලෙස AgentForge හදන app folders මෙහි තැන්පත් වේ.

වැදගත් architectural choice එක

server.py එක සියලු logic තබාගන්නේ නැහැ. server_runtime.py හි `_RUNTIME_PARTS` list එකෙන් server_modules files එකින් එක කියවා exec කර එකම global namespace එකට දමයි. ඒ නිසා old imports සහ public behavior නොබිඳී files කුඩා domains වලට බෙදා ඇත. Viva එකේ වැදගත් point එක: load order matters, මොකද පසුව load වන module එක කලින් module එකේ function/constant භාවිතා කරයි.

Architecture caution

මෙය normal Python imports පමණක් භාවිතා කරන architecture එකකට වඩා වෙනස්. Shared runtime namespace එක නිසා server_runtime.py හි order එක වැරදුනොත් name එකක් define වීමට පෙර භාවිතා විය හැක. ඒ නිසා එම file එක runtime wiring diagram එකත් dependency order එකත් දෙකම වේ.

App එක start වෙන හැටි

User start-agentforge.bat එක run කරන මොහොතේ සිට desktop window එක විවෘත වෙන තුරු.

#	පියවර	වෙන දේ
1	start-agentforge.bat	Node.js 20+ සහ Python 3.10-3.13 නිබේද බලයි; නැත්නම් private runtime download කරයි.
2	npm dependencies	desktop/ සහ studio/ packages install කර Electron සහ Next.js සූදානම් කරයි.
3	Python packages	FastAPI, WebSocket, Mongo, report/document, Playwright සහ අනෙක් backend packages සූදානම් කරයි.
4	Playwright Chromium	Real browser E2E testing සඳහා Chromium නිබේද බලයි; නැත්නම් install කරයි.
5	Electron launch	desktop/main.js ක්‍රියාත්මක කර splash window එක පෙන්වයි.
6	Port cleanup	පරණ AgentForge process එකක් අල්ලාගෙන ඇති ports පමණක් reclaim කරයි; වෙනත් app එකක් port 3000 භාවිතා කළොත් නවතී.
7	Python backend	backend/server.py start කර ports 7824, 7825, 7826 සහ 7834 සූදානම් කරයි.
8	Studio	studio/ තුළ npm run dev කර Next.js UI port 3000 මත start කරයි.
9	Desktop view	Studio answer කළ පසු Electron window එක http://localhost:3000/_agentforge වෙත load වේ.

Desktop files

File	වැදගත් functions	වගකීම
start-agentforge.bat	ensure_node, ensure_python, ensure_npm_packages, ensure_playwright	One-click installer සහ launcher.
desktop/ main.js	startBackend, startStudio, createWindow, reclaimPorts, launch	Child processes start/stop කර Electron window එක පාලනය කරයි.
desktop/ runtime.js	pythonCommand, portOpen, listenerPid, reclaimPort	Runtime හඳුනාගැනීම සහ safe process/port helpers.
desktop/ preload.js	launch, onLog, onStep	Splash page එකට limited IPC bridge එකක් ලබා දෙයි; direct Node access නොදෙයි.
desktop/ splash.html	Splash UI	Setup/start progress සහ backend logs userට පෙන්වයි.

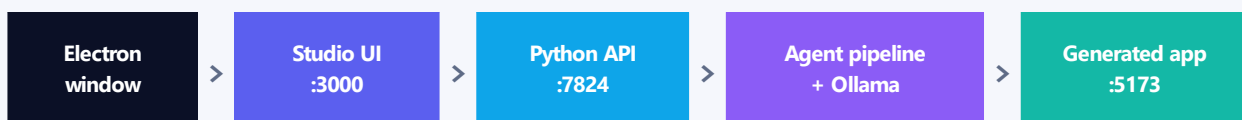
Ports සහ connections

මෙම table එක viva එකේ 'frontend-backend connect වෙනත් කොහොමද?' කියන ප්‍රශ්නයට direct answer එකයි.

Port	Service	Protocol	කාර්යය
3000	Next.js Studio	HTTP	AgentForge control-room UI. Electron මෙය load කරයි.
7824	Python API	HTTP/JSON	Projects, settings, files, images, build fallback සහ proxy endpoints.
7825	Live event channel	WebSocket	Build commands serverට; logs/progress/files/tests UI එකට live stream.
7826	SRS sidecar	HTTP	Approved plan, handoff, SRS PDF/JSON ලබා දෙන boundary.
7834	Deploy sidecar	HTTP	Deployment features සඳහා. මෙහි internal flow මෙම guide එකෙන් පිටත.
5173	Generated app	HTTP + HMR	Builder හදන app එකේ live preview සහ browser QA target.
11434	Ollama local daemon	HTTP streaming JSON	Local models; model id cloud නම් ollama.com වෙත route විය හැක.
27017	MongoDB	Mongo wire protocol	Generated app data සහ per-project databases.

Next.js rewrite එකෙන් ports දෙක එක screen එකක් වගේ පෙනෙන්නේ කොහොමද?

- studio/next.config.js හි basePath එක /__agentforge වේ. එම නිසා Studio UI තමන්ගේ namespace එකක් තුළ පවතී.
- /__agentforge/api/... requests port 7824 Python API එකට rewrite වේ.
- /__agentforge නොවන requests port 5173 generated app එකට rewrite වේ.
- PreviewPane හි iframe src="/" ලෙස නිවුණත් Studio rewrite එක නිසා iframe එකට generated app එක ලැබේ.
- WebSocket events rewrite එක හරහා නොයයි; studio/lib/ws.js direct ws://host:7825 වෙත connect වේ.



UI එක build command එක සකස් කරන හැටි

Approved SRS එකක් භාවිතා කළත් direct prompt එකක් භාවිතා කළත් අවසානයේ එකම agent_build message shape එකට යයි.

Main UI files

File	Functions/area	වගකීම
studio/ app/ page.jsx	Studio	Main screen, tabs, project open/import, edit chat, WebSocket connect.
studio/ components/ Home.jsx	begin, afterTheme, startBuild, planFirst, acceptSrs	New project prompt, SRS handoff acceptance, theme/logo flow සහ build launch.
studio/ components/ Sidebar.jsx	Model/ settings handlers	Planner, Design, Builder, QA, SRS සහ Deploy models තෝරා settings save කරයි.
studio/ components/ ThemePicker.jsx	Theme selection	Whole-app design directions ඉල්ලලා user pick එක ලබා ගනී.
studio/ components/ LogoPanel.jsx	Logo flow	Images enabled නම් logo create/upload/skip කරයි.
studio/ components/ BuildOverlay.jsx	resolveState, TaskTable	Raw events human-readable progress, task rail සහ testing state ලෙස පෙන්වයි.
studio/ components/ PreviewPane.jsx	submit, fire, swapPicture, syncPath	Iframe preview, element edit, pencil edit, image edit, undo සහ live E2E view.
studio/ components/ CodePane.jsx	save	Generated source browse/edit/save කරයි.
studio/ lib/ store.js	useStore	Zustand state: project, logs, files, phases, tests, selected models, SRS state.
studio/ lib/ ws.js	connect, send, handle	Command යවයි; live backend event එක store update එකක් බවට පත් කරයි.
studio/ lib/ api.js	req, post, localJob, srsJob	HTTP requests සහ long-running jobs poll කිරීම.

Approved SRS එකෙන් Build button දක්වා

Function	සරල අර්ථය
acceptSrs(handoffPrompt, id)	Approved handoff prompt එක text box value එකක් කර srsId එක තබා ගනී.
begin(prompt, srsId)	මුලින් visual direction තෝරාගැනීම සඳහා ThemePicker විවෘත කරයි.
afterTheme(...)	Images on නම් LogoPanel; නැත්නම් startBuild.
startBuild(...)	Store reset කර busy/progress දමා agent_build message එක සකස් කරයි.
send(...)	WebSocket open නම් 7825 හරහා යවයි; socket down නම් HTTP /agent-build fallback භාවිතා කරයි.

agent_build message එකේ වැදගත් දේ

- prompt: approved handoff text හෝ user typed brief.
- srs_id: full SRS artifacts backend එකෙන් නැවත ලබා ගැනීමට link එක.
- planner_model, design_model, builder_model: role එකකට වෙනම model තේරීම.
- qa_model: unit/E2E authoring සහ debugging model.

- think: Builder සහ QA reasoning mode; Planner සහ Design code අනුව reasoning off.
- theme, theme_html, theme_dir: user අනුමත visual contract.
- logo සහ uploads: generated project public assets වලට ගෙන යන approved media.

06

SRS handoff එකේ සිට project folder එකට

SRS internal generation නොව, approved document එක Builder විසින් භාවිතා කරන exact handoff boundary එක.

Handoff එක එක prompt එකක් පමණක් නොවේ

SRS sidecar එක builder-handoff endpoint එකෙන් structured object එකක් ලබා දෙයි. එහි compact prompt එකක් නිවුණත් requirements, pages, APIs, roles, access matrix, workflows, data models, feature contracts, testing contract, traceability, validation rules සහ acceptance criteria වෙන වෙනම machine-readable fields ලෙසත් තිබේ.

Handoff field	Builderට කියන දේ
app_name / product_intent	App එකේ නම සහ business purpose.
requirements / source_requirements	FR identifiers සහ exact approved actions.
pages	Exact routes, allowed roles, expected page file සහ page purpose.
apis	HTTP method, URL, access සහ expected route.js file.
models / relationships	Mongo collections, fields, types සහ references.
feature_contracts	Requirement එකට role, route, API, data සහ workflow mapping.
testing_contract	Unit cases, E2E journeys, seed fixtures, role accounts සහ proof.
traceability	FR එක implement කරන page/table/test හඳුනාගැනීම.
auth_contract / access	Sign-in, roles, protected routes සහ server-side authorization rules.
invariants	Placeholder නොමැති full app, persistence, toasts, routes, ObjectId rules වැනි non-negotiables.

adopt_srs() කරන දේ

backend/server_modules/srs/srs_runtime.py හි adopt_srs(srs_id, proj_dir) function එක staged SRS එක generated project එකේ .agentforge/srs folder එකට පිටපත් කරයි. Sidecar එකෙන් plan, builder handoff, interview, document, diagrams සහ PDF නැවත fetch කර missing artifacts සම්පූර්ණ කරයි.

Generated project location	අර්ථය
.agentforge/ srs/ plan.md	Human-readable approved plan.
.agentforge/ srs/ plan.json.srs	Original structured SRS plan.
.agentforge/ srs/ handoff.json	Complete machine-readable handoff.
.agentforge/ srs/ handoff.txt	Builder-facing handoff prompt.
.agentforge/ srs/ srs_latest.json	Full SRS document.
.agentforge/ srs/ interview.json	Interview decisions.
.agentforge/ srs/ diagrams/	Mermaid sources සහ SVG diagrams.

Generated project location	අර්ථය
.agentforge/ srs/ SRS_latest.pdf	Approved SRS PDF.
.agentforge/ srs/ link.json	srs_id, staging path සහ adoption timestamp.

Double protection against lost requirements

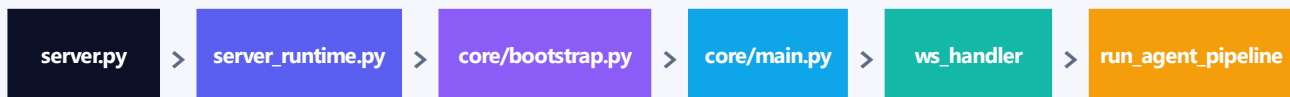
Pipeline එක prompt එක පමණක් භාවිතා නොකරයි. _srs_name_line() app name එක lock කරයි; _srs_brief() full SRS details brief එකට එකතු කරයි. එම නිසා compact handoff prompt එකේ සීමාවක් තිබුණත් full approved specification එක Builderට නැවත ලැබේ.

07

Backend command එක receive කරන හැටි

WebSocket message එක Python thread එකක් බවට පත්වී run_agent_pipeline එක ආරම්භ වන සම්බන්ධතාවය.

Server entry chain



File	Function	වගකීම
backend/ server.py	main	asyncio.run(server_runtime.main()) කර stable public endpoint එක ලේ.
backend/ server_runtime.py	_load_runtime_parts	Core, agent, QA, SRS, deploy, UI files නිශ්චිත order එකකට shared namespace එකට load කරයි.
backend/ server_modules/ core/ bootstrap.py	constants, imports, emit helpers	Ports, directories, Ollama client, Agent classes සහ live event helpers සකස් කරයි.
backend/ server_modules/ core/ main.py	main	Mongo, SRS, deploy, HTTP threads start කර WebSocket server 7825 මත තබයි.
backend/ server_modules/ agent/ builder / project_ops.py	ws_handler	agent_build message decode කර background thread එකක run_agent_pipeline අරඹයි.
backend/ server_modules/ agent/ builder / pipeline.py	run_agent_pipeline	Project එක create කිරීමේ සිට final QA සහ preview දක්වා master orchestration.

Thread එකක් භාවිතා කරන්නේ ඇයි?

Build එක මිනිත්තු ගණනක් යා හැකි නිසා WebSocket handler එක block කළොත් UI connections සහ වෙනත් requests නවතී. ඒ නිසා ws_handler daemon background thread එකක pipeline එක run කරයි. Pipeline එකේ emit() function එක main asyncio loop එකට thread-safe event එක schedule කර සියලු connected clients වෙත JSON message යවයි.

run_agent_pipeline: සම්පූර්ණ timeline

මේ project එකේ හදවත. Viva එකේ 'Build click කළ පසු මොකද වෙන්නේ?' කියන ප්‍රශ්නයට පියවරෙන් පියවර පිළිතුර.

#	Stage	වෙන දේ
1	Run safety	cancel.begin() කර previous exhausted findings clear කරයි; agent source restart අවශ්‍යද බලයි.
2	Project folder	SRS app name හෝ prompt එකෙන් safe slug එකක් හදා production-ready/<name> ඝකස් කරයි.
3	Role models	Planner, Design, Builder model ids settle කර model load/availability verify කරයි.
4	Intent receipt	.agentforge/intent.json තුළ original prompt, models, SRS id, theme සහ logo save කර resume support දෙයි.
5	Assets and SRS	Approved logo/uploads copy කර adopt_srs() සහ chosen design adoption ක්‍රියාත්මක කරයි.
6	Database startup	MongoManager.ensure_running() background thread එකක start කරයි.
7	Callbacks	Planner/Builder logs, progress, phases සහ file tokens UI events බවට map කරයි.
8	QA session	Cloud QA model තෝරා තිබේ නම් QASession එක Builder callbacks වලට bind කර file-by-file test authoring සූදානම් කරයි.
9	ArchitectAgent	Planner + Design + Builder mixins එක් object එකක initialize කර stack, Mongo URI, DB name සහ theme ලබා දෙයි.
10	Planning	make_plan() requirements වලින් plan.md සහ structured .agentforge/plan.json හදයි; missing FR/routes/capabilities තිබේ නම් replan/repair කරයි.
11	Scaffolding	Fixed Next.js/Tailwind/MongoDB/auth shell සහ config files ඝකස් කරයි.
12	Design contract	Approved theme එක deterministic CSS receipt + Design model interpretation එකක් කර design.md හදයි.
13	File generation	build_app() task order එකෙන් complete files stream කර disk එකට ලියයි; UI එකට live tokens යවයි.
14	Delivery closure	Missing planned files, capability files හෝ uncovered E2E workflows තිබේ නම් focused continuation turns දෙකක් දක්වා ලබා දෙයි.
15	Local first gates	Import, syntax, lint, symbol, client/server boundary සහ dependency checks run කර correctable issues repair කරයි.
16	Analyzer seam A	Generated source static scan කර missing files, dead links, broken imports, auth/data/action contract faults හඳුනා repair කරයි.
17	npm install	Generated project dependencies install/sync කරයි.
18	Fresh DB	Mongo available නම් previous project collections clear කර deterministic seed එකට fresh state ලබා දෙයි.
19	Production compile	npm run build run කර compiler errors evidence ලෙස Builder repair turn එකකට දෙයි; green වන තුරු bounded loop.
20	Flow convergence	Plan, routes, controls, APIs සහ data edges deterministic ලෙස එකට සම්බන්ධද බලයි.
21	Unit QA	Generated unit tests run කර failure එක test fault ද code fault ද තීරණය කර guarded repair/re-run කරයි.
22	Images finish	Background image generation/placement complete වන තුරු browser QA පෙර wait කරයි.
23	Live server	Generated Next.js app port 5173 මත start කර ready වන තුරු wait කරයි.
24	Smoke loop	TesterAgent root/pages open කර runtime, console සහ terminal faults බලයි; Bug Fixer exact evidence සමඟ repair කර නැවත serve කරයි.
25	Runtime + API gates	All pages, linked dynamic routes, credentials සහ API routes actual HTTP requests වලින් verify කරයි.
26	E2E journeys	Playwright real user flows run කර durable visible behavior prove කරයි; blocker තිබේ නම් symptom noise වැළැක්වීමට E2E skip කරයි.
27	Security + performance	Security audit සහ build serve වේ නම් performance measurement run කරයි.

#	Stage	වෙන දේ
28	Final receipt	QA JSON/report, final-flow.json සහ quality_summary event ලියයි.
29	Preview handoff	5173 answer කරනවාද බලලා clean හෝ unresolved quality state log කර edone() event UI එකට යවයි.
30	Cleanup	Selected models unload කර cancel state finish කරයි.

ඉතා වැදගත් viva distinction

Pipeline එක edone() යැවීම සහ quality_clean=True වීම එකම දේ නොවේ. Build එකේ files හදා preview serve විය හැකි නමුත් unit, E2E, runtime, API හෝ security gate එකක් red නම් app එක 'clean' ලෙස label නොකරයි. Testing tab සහ .agentforge/final-flow.json බලන්න ඔබ.

09

Planner agent එක වැඩ කරන හැටි

Planner code ලියන්නේ නැහැ. Requirements අහිමි නොවන, buildable, machine-readable map එකක් හදනවා.

Plannerගේ input

- Approved handoff prompt/full SRS brief හෝ user brief.
- Exact FR identifiers නිවේ නම් mandatory coverage ledger.
- Free-form brief නම් _source_requirements() action verbs හදුනා original action checklist එකක් සකස් කරයි.
- Operational context: logo, image resources සහ machine constraints product feature ලෙස වැරදිව ගණන් නොගැනීමට වෙනම section එකක්.
- Fixed Next.js stack rules සහ plan schema.

Plannerගේ output

- plan.md: human-readable overview, routes, pages, features සහ implementation tasks.
- .agentforge/plan.json: phases/tasks/files, dependencies, capabilities, workflows, contracts, data reads/writes, actions/testids සහ demo accounts.
- Each task හි covers list: FR/capability coverage proof.
- Each browser capability සඳහා workflow/E2E path.
- Producer-before-consumer order: data/helper/route/component/page dependencies build order එකෙන් සකස් කිරීම.

Function	කාර්යය
_source_requirements	Free-form sentence වල business action verbs බෙදා requirement checklist එකක් හදයි.
make_plan	System prompt + authoritative requirements model එකට යවා plan parse/normalize/save කරයි.
_uncovered	SRS FR id එක කිසි task එකක covers නොවී තිබේද බලයි.
_capability_gaps	Core feature එක capability නොවීම, capability එක workflow නොවීම, file mapping අසම්පූර්ණ වීම හඳුනාගනී.
_repair_requirement_coverage	Existing capability/file evidence ප්‍රමාණවත් නම් missing FR ledger id එක correct task එකට mechanical ලෙස attach කරයි.
_repair_capability_map	Capability එකට අවශ්‍ය file plan එකෙන් අනහැරී තිබේ නම් right task එකට add කරයි.

Function	කාර්යය
_unbuilt_routes	Plan එකේ සඳහන් route එක serve කරන page/route file එක task list එකේ නිවේද ලෙයි.
_normalise_plan	Model JSON එක stable AgentForge schema එකකට හැරවයි.

Planner prompt නියෝග files

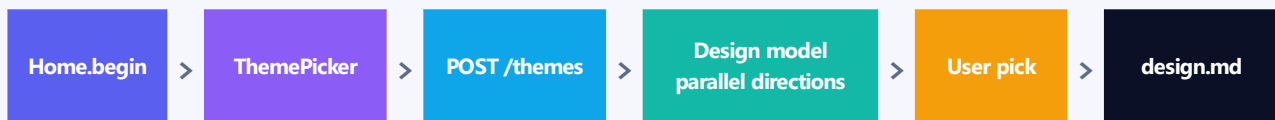
Location	කියන මූලික අදහස
backend/ agents/ planner/ prompt.py	Part A + Part B එකතු කර NEXT_PLANNER_SYSTEM export කරයි.
backend/ agents/ planner/ prompt_a.py	Request decides scope; jobs complete කරන්න; flow first; no invented features.
backend/ agents/ planner/ prompt_b.py	Exact contracts, reads/writes, actions, sections, testids, data/auth rules සහ JSON plan structure.
backend/ agents/ builder/ prompts/ stack.py	Next.js 16 App Router, JavaScript, Tailwind, MongoDB, Better Auth boundaries සහ folder shape.
backend/ agents/ planner/ execution.py	Dynamic user prompt, coverage preflight සහ replan requests සකස් කරයි.

10

Design role සහ UI contract

Design model direct React code ලියන්නේ නැහැ. Builder අනුගමනය කරන visual rules සහ exact receipt එක සකස් කරයි.

Theme selection flow



- UI /themes API එකෙන් whole-app visual directions කිහිපයක් parallel ලෙස ඉල්ලයි.
- Selected design HTML staging area එකේ save වේ; build start වන විට project .agentforge/designs folder එකට adopt වේ.
- ArchitectRuntimeMixin._design_md() chosen HTML එකෙන් deterministic CSS facts receipt එකක් ගනී.
- Design model එම HTML බලලා typography, colors, radius, spacing, borders/shadows සහ layout rules විස්තර කරයි.
- Model thin/failed answer එකක් දුන්නත් CSS receipt fallback එක නිසා chosen design facts අහිමි නොවේ.
- Builderට design.md advice නොව binding contract එකක් ලෙස ලබා දෙයි.

DESIGN_SKILL contract එකේ ප්‍රධාන නීති

- Approved color, font, radius, spacing සහ shadow values නැවත භාවිතා කරන්න.
- Tailwind utility classes, lucide icons සහ responsive small-screen-first layout.
- Every data screen: loading skeleton, useful empty state, failed state with retry.
- Forms: labels, understandable validation, saving state සහ confirmed success.
- Real buttons/links only; inert clickable div, dead navigation සහ one-off arbitrary styles නොමැති වීම.

Location	Prompt/Function
backend/ agents/ picture/ themes.py	DESIGN_SYSTEM, design_prompt සහ visual direction helpers.
backend/ agents/ builder/ orchestration/ runtime.py	DESIGN_SKILL, _design_md, start_conversation.
backend/ server_modules/ agent/ design/ stage.py	Theme staging/adoption and selected design persistence.
studio/ components/ ThemePicker.jsx	User-facing preview and pick/skip flow.

11

Builder agent එක app files ලියන හැටි

ArchitectAgent කියන්නේ එක giant class එකක් නොව, focused mixins කිහිපයක් එක object එකකට assemble කළ domain agent එකක්.

ArchitectAgent mixin structure

Mixin	වගකීම
ArchitectRuntimeMixin	Model calls, prompt assembly, context budget, design.md සහ conversation.
ArchitectMemoryMixin	Current run memory, prior attempts සහ summary.
ArchitectWriteMixin	Safe full-file writes, validation, callbacks සහ protected paths.
ArchitectPlanningMixin	Requirement extraction සහ plan creation.
ArchitectPlanNormalizeMixin	Model plan schema normalize/repair.
ArchitectScaffoldMixin / ArchitectNextScaffoldMixin	Fixed generated project shell, Mongo/auth/config files.
ArchitectBuildMixin	Planned file list, build turns සහ first-write gates.
ArchitectLedgerMixin	Symbols, data vocabulary, contracts, routes සහ capability ledgers.
ArchitectTurnMixin	Continue prompts, context snapshots, parser, tool requests සහ loop protection.
ArchitectSymbolMixin	Imports/exports/symbol closure verification.
ArchitectNextRulesMixin	Next.js client/server/auth/layout/lint fixes.
ArchitectBoundaryMixin	Server/client serializability සහ protected dependency boundaries.
ArchitectDeliveryMixin	run(), missing delivery gap closure, resume.
ArchitectPersistenceMixin	plan/conversation save, existing project reload, final output checks.

Builderගේ three-layer prompt

Layer	Location	Modelට ලැබෙන දේ
System rules	part_a.py + part_b.py + stack.py	Full files, real data/actions, exact testids/contracts, no stubs, Next.js/Mongo boundaries.
Initial user turn	start_conversation()	Idea + approved plan + design.md + continuous task instructions.
Continuation turn	_continue_prompt()	Current task, symbol ledger, house rules, source snapshot, outstanding files සහ relevant reminder.

Files stream කරන mechanism

- Model එක complete file එක <write_file path='...'> block එකක් ලෙස හෝ write_file tool call එකක් ලෙස දෙයි.
- FileStreamParser streaming chunks එකෙන් text සහ file boundaries හඳුනාගනී.
- on_file_start UI එකට stream_start event යවයි; token chunks stream event වලින් Code pane/overlay update කරයි.
- on_file_end සම්පූර්ණ content එක validate කර disk එකට ලියලා stream_end සහ file event යවයි.
- Mid-file truncation හඳුනාගන්නොත් next turn එකේ එම file එක මුල සිට complete ලෙස නැවත ඉල්ලයි.
- Model repeated text හෝ runaway output දුන්නොත් size/repetition guards turn එක නවතා usable portion එකෙන් ඉදිරියට යයි.

Why complete files, not diffs?

Agent workflow එක model-generated patch fragments එකට වඩා complete file contract භාවිතා කරයි. එමගින් stream parsing, disk truth, syntax checks, export preservation සහ rollback snapshots සරල වේ. ඒත් complete rewrite එකක් working behavior අහිමි කළ හැකි නිසා scope guards, snapshots සහ post-write gates අවශ්‍ය වේ.

12

Generated app scaffold සහ technology stack

Builder හැම app එකකටම මුලින් stable fixed shell එකක් දී පසුව business files ලියයි.

Technology	භාවිතය
Next.js 16.3 App Router	Generated pages, route handlers, server components සහ dev/production build.
React 19	Function components සහ hooks.
JavaScript only	TypeScript භාවිතා නොකර .js/.jsx extension rules.
Tailwind CSS 3.4 generated app	All styling utility classes. Studio එකේ Tailwind 4 භාවිතා වේ.
MongoDB official driver	Data persistence via generated lib/mongodb.js.
Better Auth	Roles/accounts අවශ්‍ය apps වල authentication scaffold.
lucide-react	UI icons.
framer-motion	Animation package, fixed allowed dependency.
Vitest + Testing Library	Generated unit tests.
Playwright Chromium	Real browser E2E journeys.

Scaffold කරන ප්‍රධාන files

Generated file	විගකීම
package.json	Pinned scripts/dependencies; existing valid installed deps preserve කරයි.
next.config.mjs	Next settings, dev origins, browser-to-terminal logging.
jsconfig.json	@/* alias project root එකට map කරයි.
tailwind.config.js / postcss.config.js	Styling build setup.
app/ globals.css	Tailwind layers සහ base page styles.
app/ layout.jsx	Structural html/body root; auth pagesට global navbar inject නොකරයි.
app/ page.jsx	Temporary Building placeholder; Builder real page එකෙන් replace කළ යුතුය.
lib/ mongodb.js	Connection, collections, serialization, ObjectId සහ file/GridFS helpers.
app/ api/ health/ route.js	Runtime health route.
lib/ auth.js / lib/ auth-client.js	Only when app needs authentication.
app/ api/ auth/ [...all]/ route.js	Better Auth handler, auth apps පමණි.
.env.local	Project DB URL/name සහ auth runtime values.
AGENTS.md / CLAUDE.md	Generated project එක edit කරන future agents සඳහා exact stack rules.

.js සහ .jsx rule එක

Tag/JSX අඩංගු page, layout, error සහ component files .jsx වේ. Plain modules සහ API route handlers .js වේ. එකම basename එකෙන් .js සහ .jsx දෙකම නොලියයි, මොකද bundler එක එකක් පමණක් resolve කර dead code bug එකක් හදාගත හැක.

13

Analyzer: code කියවලා plan එකට සසඳන inspector

Builder ලියූ දේ model memory එකෙන් විශ්ලාස නොකර disk එකෙන් නැවත කියවයි.

Analyzer scan කරන දේ

- Plan එකේ file එක disk එකේ missing හෝ scaffold placeholder ද?
- Local imports, named/default exports සහ packages resolve වෙනවාද?
- Filesystem routes enumerate කර dead links, dead API endpoints සහ unreachable pages තිබේද?
- Next.js async params/searchParams, server/client boundaries සහ props contracts හරිද?
- Mongo ObjectId, collection reads/writes, seed volume සහ unique-index hazards හරිද?
- Session cookie/auth role redirects සහ demo identities හරිද?
- Visible actions/testids, workflow controls සහ capability file mapping ඇත්නටම source එකේ තිබේද?
- Uploads persistent storage භාවිතා කරනවාද?
- Plan contracts හි source control, URL, method, request/response fields සහ effect දෙපැත්තම implement වෙනවාද?

backend/agents/gates file	කාර්යය
analyzer_context.py	Files/routes/contracts inventory සහ project source truth.
analyzer_code.py	Imports, exports, package සහ source-level correctness.
analyzer_data.py	Mongo/data/seed relationships.
analyzer_ui.py	Actions, testids, inert controls, ObjectId UI/data edges.
analyzer_workflows.py	Role login, workflow controls, capability mapping සහ scan assembly.
analyzer_runtime.py	Actual HTTP route probes, credentials, dynamic links සහ model-assisted diagnosis.
analyzer_repair*.py	Findings group කර evidence prompt සකස් කර scoped full-file repairs apply/verify.
agent.py	All mixins assemble කරන AnalyzerAgent facade.

Static scan සහ semantic diagnosis වෙනස

Static scan regex/structure rules වලින් දන්නා errors ඉක්මනින් හඳුනාගනී. Semantic diagnosis model එකට plan, file inventory, routes සහ already-found problems ලබා දී අවශ්‍ය files read_file requests වලින් කියවීමට ඉඩ දෙයි. Model feature promise එක ඇත්නම් complete නොවීම වැනි regex එකට අමාරු gaps සොයයි. දෙකම evidence එකක් ලෙස Finding objects බවට පත් වේ.

14

Bug Fixer: error එකෙන් root cause එකට

මෙහි repair වර්ග තුන වෙනස් කරලා තේරුම් ගන්න: compiler repair, unit-test arbitration සහ runtime/user bug repair.

A. Compiler repair

run_build_fix_loop() npm run build output එක බලයි. Missing dependency ඇත්නම් install කළ හැකි නම් install කරයි; toolchain break එකක් නම් reinstall කරයි. Application compiler error ඉතිරි නම් Builder conversation එකට exact compiler evidence සහ named source files දී affected files complete ලෙස rewrite කරයි. Accepted features, routes, testids සහ UI preserve කරන්න කියන guard එක prompt එකේ ඇත.

B. Unit-test repair arbitration

BugFixerAgent.fix() failure එක test එකේ වැරද්දක්ද app code එකේ වැරද්දක්ද තීරණය කරයි. Model මුලින් VERDICT test/code/harness/unclear එකක් දිය යුතුය. Verdict test නම් test file එක පමණක්; code නම් target source file එක පමණක් writable. Passing test cases byte-for-byte preserve කරයි. it.skip, empty assertions, expect(true) වැනි test weakening automatically reject කරයි.

C. Runtime ජීව user-reported bug repair

Function/stage	වගකීම
classify_intent	Message එක bug/feature/page/ask ලෙස route කරයි.
run_bug_report	Existing project open කර browser evidence සහ dev server logs එකතු කර transaction එකක් ආරම්භ කරයි.
_reproduce_complaint	Playwright browser එකෙන් affected route open කර fields fill/click කර console, page error, network evidence ගනී.
_infer_issue_route	Current route, complaint, console සහ source evidence වලින් correct affected route තෝරයි.
_exact_runtime_focus	Stack/location evidence නම කියන exact source files initial focus ලෙස තෝරයි.
_repair_runtime	BugFixerAgent.fix_runtime වෙත evidence-backed repair spec යවයි.
verify_after_edit	Syntax, imports, production build සහ optional runtime routes නැවත verify කරයි.
_stabilize_bug_repair	Same route නැවත නැවත reproduce කර original fault gone සහ fresh downstream fault නැති බව සනාථ කරයි.
rollback	Compile/import red හෝ live flow converge නොවූණොත් captured transaction restore කර working app එක රැකගනී.

Bug Fixer prompt locations

File	Prompt
backend/ server_modules/ agent/ repair/ chat.py	ROUTER_SYSTEM: bug/feature/page/ask intent classification.
backend/ agents/ repair/ common.py	SYSTEM: test-vs-code verdict and one-file guarded unit repair.
backend/ agents/ repair/ common.py	RUNTIME_SYSTEM: runtime stack location first, inspect dependencies, smallest evidence-backed repair.
backend/ agents/ repair/ prompt.py	Dynamic failure details, DOM, stack, harness and neighbour sources prompt.
backend/ server_modules/ agent/ repair/ build.py	Compiler evidence repair prompt generated inside run_build_fix_loop.

Core repair principle

Bug Fixer 'error message එකට සමාන වචන තියෙන page එක' අනුමානයෙන් rewrite නොකරයි. Browser/Next stack එක exact file:line location කියනවා නම් ඒක මුලින් කියවයි. Evidence root cause එක වෙනත් dependency එකක බව පෙන්වන්නේ නම් පමණක් scope expand කරයි.

QA: unit tests, browser journeys සහ final quality

Compiler green කියන්නේ code compile වෙනවා කියන එක පමණයි. QA actual product behavior prove කරයි.

Unit QA flow



- QASession Builder ලියන test-worthy files track කර manifest එක තබයි.
- UnitTestAuthor source/plan contract බලලා focused unit tests හදයි.
- TestHarness Vitest config, setup සහ Mongo/auth mocks materialise කර packages install කරයි.
- VitestRunner test report parse කර TestFailure objects හදයි.
- BugFixer test fault ද code fault ද තීරණය කර guarded repair කරයි.
- Bounded rounds තුළ progress measure කර repeated/no-progress repairs තවතයි.

E2E QA flow

- SRS testing_contract සහ Architect workflows වලින් real user journeys ගනී.
- Each journey fresh deterministic seed එකකින් පටන් ගනී; role journey එකට exact role account භාවිතා කරයි.
- E2EAgent model එක constrained grammar එකක steps ලියයි; ඒවා Playwright actions බවට normalize වේ.
- Browser console, page errors, HTTP requests/responses, screenshots සහ visible DOM evidence collect කරයි.
- Failure kind selector, behavior හෝ crash ලෙස වෙන් කර model debugger + deterministic rules root cause තීරණය කරයි.
- Repair පසු same journey නැවත run කර durable visible effect prove කරයි.
- Final clean-room pass previous browser state මත depend නොවී app එක නැවත පරීක්ෂා කරයි.

QA prompt locations

File	Role
backend/ qa_agent/ author_common.py	SYSTEM prompt for honest unit tests and approved helpers.
backend/ qa_agent/ e2e_common.py	SYSTEM prompt and auth rules for browser journey authoring.
backend/ qa_agent/ debugger_common.py	SYSTEM prompt for evidence-backed E2E diagnosis.
backend/ qa_agent/ e2e_context.py	Prompt budget and current project/journey evidence assembly.
backend/ qa_agent/ e2e_execution.py	Playwright execution and event collection.
backend/ qa_agent/ security.py	Security rules independent of the model.
backend/ qa_agent/ report_pdf.py	Testing tab/export සඳහා QA PDF report generation.

quality_clean formula එක සරලව

Final clean label එක ලැබෙන්නේ production build green, planned flow complete and conclusive, runtime routes clean, APIs clean, security major/blocker නැති, E2E run වී all journeys pass, unit tests run වී clean සහ extra runtime errors නැති විට පමණි.

Model layer: Planner, Design, Builder සහ QA models

UI එකේ model තේරීම server-side OllamaClient එකට සම්බන්ධ වන ආකාරය.

Role isolation

Role	Setting key	Usage
Planner	planner_model	Plan only; thinking code අනුව off.
Design	design_model	Theme interpretation; thinking off.
Builder	builder_model	Full app code සහ update/repair turns; think toggle බලපායි.
QA	qa_model	Unit/E2E authoring/debugging; cloud-only gate, think toggle බලපායි.
SRS	srs_model	Scope boundary; independent setting.
Deploy	deploy_model	Scope boundary; independent setting.

OllamaClient route logic

- Local model නම් default `http://localhost:11434` වෙත `/api/chat` request යවයි.
- Model id -cloud හෝ :cloud ලෙස හඳුනාගෙන API key තිබේ නම් `https://ollama.com` වෙත Bearer token සමඟ යවයි.
- Local Ollama daemon cloud model proxy එකක් ලෙස signed-in නම් එම local daemon හරහා cloud model usable වේ.
- `chat_stream()` model output chunks yield කර Builder file streaming possible කරයි.
- Model think option support නොකර 400 දුන්නොත් think field ඉවත් කර retry කරයි.
- Transient 429/5xx/timeout errors exponential backoff සමඟ retry කළ හැක.
- Context window model `/api/show metadata` සහ local override අනුව තීරණය වේ.
- Run අවසානයේ local model unload request එකෙන් VRAM නිදහස් කරයි; cloud modelට no-op.

File	වගකීම
studio/ components/ ModelPicker.jsx	Searchable picker UI.
studio/ lib/ models.js	Server catalog normalize, labels, cloud/local flags සහ context helpers.
studio/ components/ Sidebar.jsx	Role choice localStorage සහ backend settings දෙකේම persist කරයි.
backend/ agents/ core/ ollama_client.py	Discovery, settings, route, stream, retry, context, pull/unload.
backend/ server_modules/ core/ bootstrap.py	default_planner_model/default_design_model/default_builder_model සහ shared client.

Privacy/architecture point

Local model තෝරාගන්නා විට prompts local Ollama daemon එකට යයි. Cloud model තෝරා API key භාවිතා කළොත් prompt, plan/source context සහ evidence cloud endpoint එකට යා හැක. Viva එකේ local-first කියන එක cloud never කියන අර්ථය නොව, userට role-by-role routing තෝරාගන්න පුළුවන් කියන අර්ථයයි.

MongoDB සහ generated app data flow

Studio database එකක් හදන්නේ නැහැ; generated app එකේ real data persistence සඳහා per-project Mongo database ලබා දෙයි.

MongoManager responsibility

- Installed MongoDB හෝ configured MongoDB URI හඳුනාගෙන running/available state තබයි.
- Project name එකෙන් safe database name එකක් හදයි.
- Build start වෙද්දී Mongo process background එකේ සූදානම් කරයි.
- Fresh build/E2E සඳහා project database collections reset කර seed නැවත run කිරීමට ඉඩ දෙයි.
- Generated .env.local එකට MONGODB_URI සහ MONGODB_DB values ලබා දෙයි.

Generated app data request example - code නැවුම්



Client component එක MongoDB import නොකරයි. It fetches an API. Route handler එක session/role validate කර input ids ObjectId බවට convert කර official driver භාවිතා කර persistent write එක කරයි. Success response එක ලැබුණු පසු UI row/total/page state visibly update කර success toast පෙන්වයි. Error නම් error toast සහ useful error state පෙන්වයි.

Data domain files

backend/agents/data file	කාර්යය
mongo_common.py	Constants, safe DB names, URI overrides සහ base manager state.
mongo_install.py	Mongo binary/package setup.
mongo_lifecycle.py	Start, stop, availability සහ process lifecycle.
mongo_data.py	Project reset, seed execution සහ database operations.
mongo.py	Mixins assemble කරන MongoManager facade.
seed_keys.py	Generated seed image/data stable keys derive කරයි.

Live UI feedback: backend event එක screen එකට

Agent වැඩ කරන අතර userට log, progress, file සහ test updates දර්ශනය වන්නේ event contract එක නිසා.

Event type	store action	UI meaning
log	addLog	Human-readable INFO/WARN/ERROR timeline.
project	bumpProjects	New project sidebar refresh.
step	setStep	plan/install/build/serve/test status.
progress	setProgress	Percent + label; milestone model only forward moves.
phase	upsertPhase	Planner/Builder/repair task progress rail.
stream_start	liveFile + empty liveBuf	Code view follows currently written file.
stream	queueStream	Tokens 32ms batches වලින් UI render pressure අඩු කර append කරයි.
stream_end	putFile	Complete file store එකට commit කර live buffer clear.
file	putFile	Finished file content/update.
test_start / test_run	testStart / testRun	Testing state and attempt number.
test_result	testResult	Pass/fail/warn rows සහ totals.
test_fixing	testFixing	Repair round සහ errors.
e2e_event	setE2eLive	Current journey route/frame/action visible previewට mirror.
ask	question	Scope ambiguous නම් build/edit pause කර user choice ලබා ගනී.
done	busy false	Run end, project set, QA cache invalidate, sidebar refresh.
cancelled	reset + warning	Cancellation error ලෙස නොව deliberate stop ලෙස පෙන්වයි.
error	busy false + error log	Unrecoverable run error.

BuildOverlay raw logs පමණක් නොපෙන්වන්නේ ඇයි?

BuildOverlay.resolveState() liveFile, tests, E2E event, phases, recent logs සහ progress එකට බලලා Planning, Building, Repairing, Testing හෝ Finishing යන human state එක තීරණය කරයි. එමගින් low-level messages userට overload නොකර useful explanation එකක් ලබා දෙයි.

Generated project එකේ files තියෙන තැන්

Viva එකේ 'output කොහේද?' කියන ප්‍රශ්නයට exact answer.

Default path

Generated apps

Default location: backend/production-ready/<project-name>/. AGENTFORGE_PROJECTS environment variable එක set කර තිබේ නම් වෙනත් absolute/relative folder එකක් භාවිතා කරයි. OneDrive performance issue තිබේ නම් outside-OneDrive project directory එකක් තෝරාගන්න code එක warning දෙයි.

Location	අර්ථය
backend/ production-ready/ <name>/ app/	Next.js pages සහ app/api route handlers.
backend/ production-ready/ <name>/ components/	Reusable සහ client UI components.
backend/ production-ready/ <name>/ lib/	Mongo, auth, seed සහ helpers.
backend/ production-ready/ <name>/ public/	Logo, uploaded/generated images සහ public assets.
backend/ production-ready/ <name>/ tests/ unit/	QA-generated Vitest tests.
backend/ production-ready/ <name>/ tests/ e2e/	Playwright journey artifacts/tests.
backend/ production-ready/ <name>/ .agentforge/ plan.json	Machine build plan.
backend/ production-ready/ <name>/ .agentforge/ convo.json	Trimmed persistent Builder conversation for resume/edit memory.
backend/ production-ready/ <name>/ .agentforge/ intent.json	Original build request and chosen models/resources.
backend/ production-ready/ <name>/ .agentforge/ srs/	Approved SRS adoption receipt and documents.
backend/ production-ready/ <name>/ .agentforge/ designs/	All design directions and chosen HTML.
backend/ production-ready/ <name>/ .agentforge/ qa/	Manifest, test results, evidence and Test_Report.pdf.
backend/ production-ready/ <name>/ .agentforge/ final-flow.json	Final quality receipt and unresolved findings.
backend/ logs/	Server logs, image staging සහ undo snapshots.
~/ .agentforge/ settings.json	Ollama host/API key hint, role model choices, Mongo URI, image and deploy settings.
~/ .agentforge/ lessons.json	Across-project repair lessons; frequent proven remedies පමණක් promptට නැවත එක් කරයි.

Agent folder එකේ සම්පූර්ණ map එක

ඔබ කළ backend/agents refactor/domain structure එක viva එකේ පැහැදිලි කිරීමට file-by-file responsibility map.

core/ - shared infrastructure

File	වගකීම
agent_memory.py	Per-agent run notebook සහ prompt memory summaries.
build_cache.py	Source fingerprint green නම් unnecessary production build skip.
cancel.py	Cancellation state, cleanup receipt සහ safe checkpoints.
commands.py	Agent-requested commands bounded ලෙස run කර events/logs ලබා දෙයි.
docsindex.py	Installed Next.js docs index සහ model read_docs service.
nextdocs.py	Compiler/runtime errorට official installed Next docs guidance match කරයි.
nextmcp.py	Next tooling helpers/integration boundary.
ollama_client.py	Local/cloud model discovery, chat/stream, retry, context සහ settings.
source_guidance.py	Human-readable source comments සහ image requirement guidance.
workspace.py	Modelට safe list/read/search project tools සහ TOOL_HELP contract.

builder/prompts/ - model rules

File	වගකීම
builder.py	Builder prompt parts එකතු කරන facade.
catalog.py	Stack nameට planner/builder/rules/source roots registry.
part_a.py	First-write acceptance, real data/actions, tests සහ runtime preflight.
part_b.py	Continuation quality, Mongo/auth/UI conventions සහ output format.
stack.py	WRITE_FILE_TOOL සහ fixed Next.js technology rules.

builder/orchestration/ - live model conversation

File	වගකීම
agent.py	ArchitectAgent mixins assemble කරන public facade.
common.py	Shared imports/constants for builder mixins.
core.py	FileStreamParser, token budgets, truncation and repeated-output guards.
memory.py	Conversation summaries and model memory stats.
persistence.py	plan/convo/intent load-save, update and output verification.
runtime.py	Model calls, local/cloud context, prompts, design.md and first conversation.
turns.py	Continue turns, snapshots, outstanding files and workspace tool loop.
writes.py	Safe file write, route method checks, callbacks and ownership.

builder/scaffolding/ - fixed project shell

File	වගකීම
base.py	Dependency allowlist, fallback plan, auth need detection and scaffold rules.
next.py	Actual Next.js/Mongo/Better Auth/config/public asset files.

builder/workflow/ - deterministic gates

File	වගකීම
boundaries.py	Server/client, serializable props, imports and protected dependency boundaries.
build.py	Planned file briefs, current task, build_app and generated-file first pass.
delivery.py	End-to-end run, delivery gap closure, resume and dependencies.
ledgers.py	Capability, route, data vocabulary, contract and house ledgers.
rules.py	Auth config, layout, symbols, fields, lint and Next-specific corrections.
symbols.py	Import/export symbol map and dependency closure.

21

Planner, gates, repair සහ testing files map

Agent folder එකේ remaining domain files සහ ඔවුන් එකිනෙකට connect වන ආකාරය.

planner/

File	වගකීම
app_vocab.py	App-specific nouns derive කර generic prompt examples current domain words වලට fit කරයි.
execution.py	Requirement extraction, plan call, coverage repair සහ replan logic.
normalization.py	Model plan stable schema එකකට normalize කර files/contracts/actions clean කරයි.
prompt.py	NEXT_PLANNER_SYSTEM facade.
prompt_a.py	Scope, complete jobs, flow-first architecture rules.
prompt_b.py	Machine plan schema, contracts, data, actions and E2E coverage.
refiner.py	App/site type classification සහ strategy refinement.

gates/

File	වගකීම
agent.py	AnalyzerAgent facade.
analyzer_common.py	Finding/AnalyzerReport types, regex/constants.
analyzer_context.py	Source inventory, routes, links and contracts.
analyzer_code.py	Code/import/export checks.
analyzer_data.py	Mongo/seed/data checks.
analyzer_ui.py	UI action/testid/inert control/data boundary checks.
analyzer_workflows.py	Workflow, role and capability checks; scan assembly.
analyzer_runtime.py	Live HTTP probes and semantic diagnosis.
analyzer_repair.py	Repair mixin facade.
analyzer_repair_run.py	Scan-repair-rescan convergence loops.
analyzer_repair_findings.py	Findings group/priority/repair plan.
analyzer_repair_apply.py	Model writes validate/apply and unresolved evidence check.
exports.py	Import/export checker facade.
exports_common.py	Strip strings/comments and shared parsing structures.
exports_parse.py	Import/export statements parse and local resolution.
exports_checks.py	Named/default import correctness checks.
exports_syntax.py	Node parser-backed JavaScript syntax checks.
security.py	Gate-level security checks facade/rules.

repair/

File	වගකීම
agent.py	BugFixerAgent facade.
common.py	Unit/runtime system prompts, verdict types and guard constants.
scope.py	Editable paths, prior mechanical diagnosis and import evidence.
prompt.py	Failure-specific prompt, DOM/stack/harness context and test weakening guards.
apply.py	fix/fix_runtime execution and safe commit logic.
runtime_spec.py	Runtime evidence -> exact repair files/context.
lessons.py	Cross-project proven repair lessons store and prompt block.

testing/

File	වගකීම
tester.py	TesterAgent facade.
tester_common.py	Browser stack settings, callback events and base state.
tester_browser.py	Smoke browser execution and console/page error capture.
tester_routes.py	Routes, login and page sweep logic.
reproduce.py	User complaint එකේ control locate/fill/click and before/after evidence.

22

Feature, picture, selection සහ server modules map

Build එකෙන් පසු user edits සහ server orchestration domain files.

feature/

File	වගකීම
agent.py	FeaturesAgent facade.
common.py	FeatureSpec, safe paths and base project source access.
planning.py	Requested new feature dependency-aware change plan.
apply.py	Scoped writes and validation.
audit.py	Feature touched flows/routes/actions completeness audit.
action_ids.py	Stable route/action data-testid names and workflow mapping.

picture/ + pencil/ + selection/

File	වගකීම
picture/ images.py	Image generation/download/place domain helper.
picture/ photos.py	Photo providers and safe asset retrieval.
picture/ themes.py	Whole-app design directions and theme interpretation prompts.
picture/ theme_contract.py	Chosen HTML CSS facts deterministic receipt.
pencil/ capture.py	Drawn region screenshot/context capture.
selection/ picker.py	Clicked DOM element -> source candidate, scope guard and route mapping.

server_modules/agent/

File	වගකීම
builder/ pipeline.py	Master build + QA orchestration.
builder/ project_ops.py	Project list/read/save/delete and WebSocket command router.
repair/ build.py	Compile repair loop, toolchain repair and project open helpers.
repair/ chat.py	Intent router, bug report, questions and post-edit verification.
feature/ actions.py	Feature transaction, tests, image/element edits, prompt tuning and undo.
design/ stage.py	Design staging/adoption.
picture/ images.py	Build-time image stage and uploads.
pencil/ page.py	Pencil/page update orchestration.
selection/ scope_map.py	Selected element to editable source scope.

server_modules/qa/

File	වගකීම
unit_support.py	QASession callbacks/results.
unit_stage.py	Unit test run/repair/report.
e2e_prepare.py	Seed/accounts/routes warmup before journeys.
e2e_policy.py	Round/time budgets.
e2e_stage.py	Journey execution, diagnosis, repair and convergence.
e2e_final.py	Final clean-room run.
runtime_repair.py	Runtime failure focus and BugFixer bridge.
verification.py	Security, flow, runtime, API and performance gates.

23

Prompt inventory: model එකට මොන තැනින් මොන instructions ද?

Viva එකේ 'AI apply කළේ කොහොමද?' කියන ප්‍රශ්නයට prompt chain එක කියන්න.

Prompt	Location	Purpose
SRS build handoff	backend/ srs-agent/ .../ generators/ builder_prompt.py	Approved FRs, routes, data, auth, APIs, feature/test contracts සහ definition of done.
Planner system	backend/ agents/ planner/ prompt_a.py + prompt_b.py	Request scope, complete jobs, plan schema, contracts, actions, workflows.
Stack rules	backend/ agents/ builder/ prompts/ stack.py	Next.js 16, JS, Tailwind, Mongo, auth and folder/extension rules.
Planner dynamic user prompt	backend/ agents/ planner/ execution.py::make_plan	Authoritative requirements, FR ledger, original actions and first-round preflight.
Builder system	backend/ agents/ builder/ prompts/ part_a.py + part_b.py	Whole files, no stubs, real persistence, UI states, testids, boundaries.

Prompt	Location	Purpose
Builder initial turn	backend/ agents/ builder/ orchestration/ runtime.py::start_conversation	Idea, approved plan, design.md and continuous task behavior.
Builder continue turn	backend/ agents/ builder/ orchestration/ turns.py::_continue_prompt	Current task, ledgers, source snapshot and remaining files.
Design interpretation	backend/ agents/ picture/ themes.py + runtime.py::_design_md	Chosen HTML -> binding visual rules.
Analyzer semantic diagnosis	backend/ agents/ gates/ analyzer_runtime.py::diagnose	Plan/files/routes/static findings; model may request source reads.
Analyzer repair	backend/ agents/ gates/ analyzer_repair*.py	Grouped findings + exact source + allowed repairs + evidence proof.
Compile repair	backend/ server_modules/ agent/ repair/ build.py	npm compiler evidence and implicated sources.
Intent router	backend/ server_modules/ agent/ repair/ chat.py::ROUTER_SYSTEM	bug/feature/page/ask classification.
Unit Bug Fixer	backend/ agents/ repair/ common.py::SYSTEM	test/code/harness/unclear verdict and one-file guarded repair.
Runtime Bug Fixer	backend/ agents/ repair/ common.py::RUNTIME_SYSTEM	Exact stack location, dependencies and smallest evidence-backed repair.
Unit test author	backend/ qa_agent/ author_common.py::SYSTEM	Honest tests with supported mocks/helpers.
E2E author	backend/ qa_agent/ e2e_common.py::SYSTEM	Walkable real user flow in constrained grammar.
E2E debugger	backend/ qa_agent/ debugger_common.py::SYSTEM	Browser/network/DOM evidence -> root cause verdict.
Prompt tuner	backend/ server_modules/ agent/ feature/ actions.py::tune_instruction	User edit request current project/route/element facts වලට ground කරයි.

AI apply කිරීමේ නිවැරදි explanation එක

මෙහි AI එකට 'app එක හදන්න' කියලා එක prompt එකක් පමණක් යවන්නේ නැහැ, Requirements contract එක, fixed technology rules, structured plan schema, visual design receipt, current task, disk source snapshot, exact compiler/test/browser evidence සහ safe writable scope වෙත් වෙත් prompt layers ලෙස යවයි. Model creative generation කරන අතර Python deterministic code එක coverage, routes, imports, tests, rollback සහ final quality පාලනය කරයි.

Error සහ edge cases

System එක fail වුණාම ඉතා සරලව මොකද වෙන්නේ කියලා දැනගන්න.

Situation	Behavior
Port 3000 busy	Electron වෙනත් program එක මරන්නේ නැහැ; userට close කර restart කරන්න කියයි.
Model missing	ensure_model local model pull කිරීමට උත්සාහ කරයි; නොහැකි නම් role model load error දිශේ run නවතී.
Cloud overload/timeout	Transient error නම් bounded retry/backoff; permanent error නම් stage failure.
Planner JSON unusable	Fallback plan map භාවිතා කරයි; quality lower විය හැකි නිසා warning log.
Missing files after Builder	close_delivery_gaps focused turns; still missing නම් partial delivery refuse කර build stage fail.
Mongo unavailable	Files generate වේ; data pages error විය හැකි බව warning. DB-dependent test evidence filtered carefully.
npm install fail	Install stage error සහ pipeline end; project files disk එකේ ඉතිරි වේ.
Compiler never green	Bounded rounds පසු build_ok false; preview උත්සාහ කළ හැකි නමුත් quality clean නොවේ.
Runtime repair no progress	Same fault signature repeated නම් loop නවතයි; endless rewriting නොකරයි.
E2E upstream blockers	Runtime/API hard fault නිවේ නම් E2E skip කර upstream gate reason report කරයි.
Feature/bug edit regression	Transaction snapshot restore කර previous working app නැවත serve කරයි.
User cancels build	Generated project සහ linked staged SRS cleanup කර cancelled event යවයි.
Studio socket down	Supported commands සඳහා HTTP fallback endpoint භාවිතා කරයි; live streaming අඩු විය හැක.

Viva questions සහ short answers

පිළිතුරු කටපාඩම් නොකර, bold idea එක ඔබේ වචනවලින් කියන්න.

Question	Short answer
මෙම project එකේ main purpose එක මොකක්ද?	Approved requirements වලින් full-stack app plan, generate, test, repair සහ preview කිරීම.
Frontend technology මොකක්ද?	Studio සහ generated app දෙකම Next.js/React. Studio Next.js 16 + React 19 + Tailwind 4; generated app Next.js 16.3 + React 19 + Tailwind 3.4.
Backend technology මොකක්ද?	Python async WebSocket + ThreadingHTTPServer orchestration; SRS/Deploy sidecars FastAPI-based packages.
Desktop app එක මොකෙන්ද?	Electron. එය Python backend සහ Next.js Studio processes start කර window එකේ UI load කරන shell එක.
Agent code එක start වෙන්නේ කොහෙන්ද?	UI agent_build message -> project_ops.py ws_handler -> pipeline.py run_agent_pipeline.
Planner සහ Builder වෙනස?	Planner code නොලියා tasks/files/contracts/workflows map කරයි. Builder plan/design අනුව complete files ලියයි.
Design model එක කරන්නේ?	Chosen theme HTML කියවා exact visual facts + interpreted rules design.md ලෙස Builderට දෙයි.

Question	Short answer
Builder එක giant class එකක්ද?	Public ArchitectAgent එක mixins ගණනාවක් assemble කරන facade එකක්; planning, runtime, writing, ledgers, rules, delivery වෙන වෙනම files වල.
SRS handoff lost නොවෙන්නේ කොහොමද?	srs_id මගින් full artifacts adopt කර handoff.json/txt, plan, SRS document සහ _srs_brief එක Builder inputට යොදයි.
AI model connect වෙන්නේ කොහොමද?	OllamaClient /api/chat වෙත HTTP requests. Local models 11434; cloud ids API key නිවේ නම් ollama.com.
Streaming කියන්නේ?	Model response token chunks FileStreamParser හඳුනා backend events ලෙස WebSocket හරහා UI live buffer එකට යවයි.
Generated files කොහේද?	Default backend/production-ready/<project>. AGENTFORGE_PROJECTS set කළොත් එහි.
Database එක මොකක්ද?	MongoDB official driver; generated lib/mongodb.js එක through server pages/routes භාවිතා කරයි.
Client component MongoDB direct use නොකරන්නේ ඇයි?	DB secret/server capability browserට නොයවන්න; client API route එකක් fetch කළ යුතුය.
Better Auth හැම app එකකටමද?	නැහැ. Plan එකට roles/sign-in අවශ්‍ය නම් පමණක් auth scaffold writes.
Compiler green නම් app completeද?	නැහැ. Compile success syntax/build gate පමණයි; runtime, API, unit, E2E, security සහ flow gatesත් pass විය යුතුයි.
Analyzer කරන්නේ?	Disk source සහ accepted plan සසඳා missing files, routes, imports, data/auth/action/workflow defects සොයයි.
Bug Fixer evidence කියන්නේ?	Compiler output, exact stack frame, browser console/page error, network response, DOM screenshot සහ failing test data.
Unit test fail උනාම app code හැමවිටම වෙනස් කරනවාද?	නැහැ. BugFixer test/code/harness/unclear verdict දෙයි; wrong test නම් test එක පමණක් fix කරයි.
Test cheating වළක්වන්නේ?	skip/todo, deleted assertions සහ meaningless expect patterns reject කර passing cases preserve කරයි.
E2E testing මොකෙන්ද?	Playwright Chromium. Real role accounts, routes, forms, API calls සහ visible final states භාවිතා කරයි.
Why fresh seed per E2E?	Journey එකක් පෙර journey state එකට depend නොවී deterministic සහ repeatable වීමට.
Repair scope කොහොමද control කරන්නේ?	Evidence-backed initial files, safe_change_path/editable rules, protected files, import checks සහ transaction rollback.
Feature edit fail උනාම?	Before snapshot restore කර previous app state නැවත serve කරයි.
WebSocket එකේ කාර්යය?	Commands යවීම සහ live logs/progress/file/test events ලබා ගැනීම.
HTTP API එකේ කාර්යය?	Settings, project files, images, jobs, SRS/deploy proxy සහ socket fallback.
Studio preview 5173 පෙන්වන්නේ කොහොමද?	Studio next.config rewrite /_agentforge නොවන routes 5173 වෙත proxy කරන නිසා iframe '/' generated app එකයි.
quality_clean condition එක?	Build, flow, runtime, API, security, E2E, unit සියල්ල clean සහ runtime errors zero.
done event සහ clean sameද?	නැහැ. Run end වී preview නිවුණත් unresolved quality gate තිබිය හැක.
Resume support කොහොමද?	intent.json, plan.json සහ convo.json disk එකේ නිසා missing planned files හඳුනා same project continue කරයි.
Agent memory එකේ purpose?	Earlier attempts/lessons contextට ගෙන repeated mistakes අඩු කරනවා; disk source truth එකට වඩා memory ඉහළ නොවේ.
Why server_runtime load order matters?	Modules එක shared namespace එකකට exec වන නිසා later functions කලින් definitions මත depend වේ.
No deployment scope කියන්නේ?	මෙම build explanation 5173 live preview සහ final quality receipt දක්වා; cloud publish logic වෙනම deployment-agent domain.

1-minute සහ 3-minute viva scripts

ඔබට අදාළ තරමට technical detail එක වැඩි කර කියන්න පුළුවන් scripts.

1-minute answer

කියන්න

මම කළ agent/server කොටස AgentForge එකේ approved requirement එක full app එකක් කරන orchestration layer එකයි. UI එක agent_build message එක WebSocket 7825 හරහා backendට යවනවා. ws_handler එක run_agent_pipeline background thread එකක start කරනවා. Pipeline එක SRS handoff එක project එකට adopt කර Planner, Design සහ Builder models set කරනවා. Planner plan.md සහ plan.json හදනවා. ArchitectAgent scaffold එක දාලා plan එකේ tasks අනුව Next.js/React/Tailwind/MongoDB files complete ලෙස stream කර ලියනවා. Analyzer plan එකට source එක compare කරනවා. npm build, unit tests, runtime/API checks සහ Playwright E2E run වෙනවා. Fault එකක් ලැබුණොත් BugFixer exact compiler/test/browser evidence බලලා safe files පමණක් repair කර නැවත verify කරනවා. අවසානයේ app එක localhost:5173 මත preview කර final quality receipt එක ලියනවා.

3-minute answer structure

- Start: desktop launcher -> Electron -> backend ports + Studio port 3000.
- UI: Home.startBuild payload, role-specific model selection, WebSocket + HTTP fallback.
- SRS boundary: srs_id + handoff prompt; adopt_srs full artifacts .agentforge/srs වෙත.
- Planner: authoritative FR/action ledger -> pages/routes/tasks/files/contracts/capabilities/workflows.
- Design: approved theme -> CSS receipt + design.md.
- Builder: fixed Next stack scaffold -> complete full-file streaming -> ledgers/snapshots/continuation turns.
- Gates: import/syntax/analyzer -> npm build repair -> unit QA -> live server -> runtime/API -> E2E -> security/performance.
- Bug Fixer: reproduce first, verdict/scope guard, transaction rollback, re-run until stable.
- Output: production-ready project, .agentforge receipts, preview on 5173; done vs clean distinction.

Glossary - ඉතා සරල අර්ථ

Technical word එකක් ඇහුවොත් භයානක නොවී සරලව කියන්න.

Word	සරල අර්ථය
Agent	ලබා දුන් role/rules අනුව model එක භාවිතා කර task එකක් කරන software worker.
Orchestration	Workers සහ tools හරි order එකට connect කර වැඩේ අවසානය දක්වා ගෙන යාම.
Prompt	Model එකට role, rules, context සහ required output කියන instruction package එක.
SRS handoff	Customer-approved requirements Builderට lossless ලෙස දෙන contract.
Capability	Userට ඇත්තටම කරන්න පුළුවන්, proof තිබෙන business ability එක.
Contract	Files දෙකක් අතර URL/method/fields/effect එකම වචන වලින් එකඟ කරන machine rule.
Scaffold	App එකේ fixed foundation/config/base helpers.
App Router	Folder/file name එකෙන් URL route තීරණය කරන Next.js routing system.
Server Component	Server එකේ run වී DB/session access කළ හැකි React component.
Client Component	Browser එකේ run වී click/state හැසිරවිය හැකි component.
API route	Browser request එක validate කර database/business operation කරන server endpoint.
WebSocket	Connection එක open තබා commands සහ live events දෙපැත්තට යවන channel.
Streaming	Model answer එක අවසානය තෙක් නොසිට token/file progress live ලැබීම.
Static analysis	App run නොකර source structure බලලා defects සොයන checks.
Runtime verification	App run කර pages/APIs actual requests වලින් පරීක්ෂා කිරීම.
Unit test	කුඩා function/component/route behavior එක focused ලෙස පරීක්ෂා කිරීම.
E2E test	User journey එක browser එකෙන් මුල සිට අවසානය දක්වා කිරීම.
Evidence-backed repair	Guess නොකර stack/test/browser/network proof අනුව fix කිරීම.
Rollback	New change එක වැරදුනොත් snapshot එකෙන් පරණ working state නැවත ගෙන ඒම.
Convergence	Repair round එකෙන් errors අඩුවී stable correct state එකකට ළඟා වීම.
Quality gate	Pass විය යුතු objective check එක; fail නම් clean label නොලැබේ.
ObjectId	MongoDB record identifier type; browser string එක server boundary එකේ convert කළ යුතුය.
Seed data	Demo/testing සඳහා repeatable initial database records.

අවසාන මතක පත්‍රය

Viva එකට යාමට පෙර මේ page එක දෙවරක් කියවන්න.

මතක තබාගන්න points 12

- Start chain: start-agentforge.bat -> Electron -> backend server.py + Studio Next.js.
- Build chain: Home.startBuild -> ws.send(agent_build) -> ws_handler -> run_agent_pipeline.
- Approved SRS is both prompt + structured files; adopt_srs preserves full truth.
- Planner decides what/files/order/contracts; Builder writes implementation.
- Design role turns approved visual choice into binding design.md.
- ArchitectAgent is a mixin-based facade, not one unmanageable file.
- Builder writes complete files via FileStreamParser and live WebSocket events.
- Generated app stack: Next.js 16, React 19, JS, Tailwind, MongoDB, optional Better Auth.
- Analyzer reads disk truth and compares against plan, routes, data and workflows.
- Bug Fixer reproduces first, uses evidence, scopes writes, verifies and can rollback.
- QA uses Vitest unit tests and Playwright real journeys; QA model is cloud-only in current code.
- Preview ready is not automatically clean; quality_clean needs every major gate pass.

Final one-line answer

AgentForge එකේ strength එක model එක code ලියන එක පමණක් නොව, approved requirements -> structured plan -> controlled file generation -> deterministic checks -> real browser proof -> bounded repair -> final receipt කියන closed loop එකයි.

Review basis

මෙම guide එක current workspace හි studio, desktop, backend/server_runtime.py, server_modules, backend/agents, qa_agent සහ SRS handoff generator files කියවා සකස් කරන ලදී. Source code edit කිරීමක් සිදු නොකළ අතර PDF artifact එක පමණක් output/pdf යටතේ නිර්මාණය කර ඇත.